DAZzle Bros

DAZ MY CAR!

Software Project Initiation Document

PROJECT TEAM

Zachary Zaslau

Anthony Toscano

Daniel De La Torre

Delivery: November 6, 2026

October 26, 2026 | Version 0.2

One plan. Clear responsibilities.

DAZzle Bros / Delivery: November 6, 2026 / Internal implementation guide

Purpose. DAZ MY CAR! searches a historical used-car dataset locally to find listings within a strict budget and eight required preferences. Return ten results per page, with exact matches first and clearly explained compromises afterward. Target all approximately 3 million source rows from the roughly 10 GB CSV; no deliberate sample cap in the release.

Area	Baseline / status
Frontend - proposed	React + TypeScript + Vite + Tailwind CSS. Motion optional after correctness. Vite fits a local browser app with a separate Python backend.
Backend - proposed	Python + FastAPI + Pydantic. NumPy stores compact fields; it must not replace assessed algorithms. Local files; no database server.
Structures - confirmed	Custom B-tree indexed by price, followed by matching and a custom heap-backed priority queue. Comparison pair remains unresolved.
Ownership - confirmed	Zachary Zaslau: frontend/integration. Anthony Toscano: B-tree/data preparation. Daniel De La Torre: heap/matching. All: review, comparison, tests and handoff.
Runtime - confirmed	2020 M1 MacBook and newer computers. Minimum RAM, actual CSV schema and dependency versions still need confirmation.

Scope and acceptance

Required: questionnaire, B-tree budget retrieval, eight-check scoring, 8/8-6/8 tiers, pagination, make/model refinement, listing details/images, reproducible full-data preparation, tests and performance analysis. Done means a teammate can set up a fresh machine, process the full source, run the website, reproduce checks and explain the algorithms.

Excluded initially: accounts, payments, live scraping, maps, cloud deployment, ML recommendations and guaranteed live listings. Images may require network access; provide local placeholders. Do not describe the snapshot as current inventory.

Decision labels and course compliance

Confirmed = agreed behavior or supplied course requirement. Proposed = this PID's engineering default. Open = team decision required. The course requires two comparable, non-trivial implementations from scratch and at least 100,000 data points. A B-tree price filter plus heap ranker alone performs different tasks; page 5 defines the open comparison gate.

Version 0.2 condenses the original plan. Pages 3-5: behavior, data and algorithms; 6-8: API, folders, setup and tests; 9-10: analysis, delivery and open decisions. This internal PID is separate from the course proposal's two-page cap.

Eight preferences. Clear matches.

Confirmed matching rules; proposed tie-breaking and refinement defaults.

Input	Pass condition
Budget - hard limit	Inclusive minimum/maximum USD, stored as integer cents. Out-of-budget cars are always excluded; budget earns no score point.
Year / mileage	Year within inclusive min/max range; mileage in miles <= selected maximum. Each category earns at most one point.
Body / fuel / transmission	Each normalized value belongs to its selected non-empty set.
Drivetrain / color	Each normalized value belongs to its selected non-empty set; color means exterior color family.
Seating	Maximum seating >= required minimum seats.

```
matched_count = sum(eight_boolean_checks)
include = within_budget and matched_count >= 6
score_percent = matched_count * 12.5
rank_key = (-matched_count, price_cents,
            mileage_or_infinity, record_id)
```

All eight categories are mandatory; no Skip or Any. Unknown data fails that check. Selecting black OR blue remains one color check. No partial credit for near misses. Show 8/8 (100%), then 7/8 (87.5%), then 6/8 (75%); exclude 5/8 and lower. Proposed ties: lower price, lower mileage, stable ID. Never rewrite preferences or relax budget.

User flow

Landing > Budget > Preferences > Review > Searching > Results > Details. Retain the prebrief's guided style, using the agreed inputs instead of its wheel/make steps. Preserve values on Back/Edit; validate required choices, nonnegative finite numbers and ordered ranges in both frontend and backend.

Paginate one ranked list, ten cars per page. At a tier boundary, insert an Exact/Closest heading without duplicating or padding cards; every card shows its count and missed/unknown fields. Make/model refinement applies to the entire eligible set before pagination, resets page 1 and updates counts. Model choices are alphabetic; car cards remain ranked. Clear incompatible model selections.

Empty, loading and failure states

If no exact matches: **"None match all your preferences. Here are the closest matches."** If nothing reaches 6/8: **"No cars match at least 6 of your 8 preferences within your budget."** Distinguish no budget candidates, empty refinement and technical failure. Preserve input, show retry, and disable Next after the last result.

Keep the cars. Reduce the overhead.

Anthony leads preparation; all members verify the full-data release.

Do not load the 10 GB CSV into browser memory or millions of Python dictionaries. Prepare it once offline; runtime opens compact read-only files and fetches listing details only for the visible page. Preserve every valid listing, including repeated external IDs; assign a contiguous internal record_id.

Stage	Implementation / exit artifact
Inspect	Read headers and bounded samples with csv. Verify real column names, units, IDs, image/listing fields and license. Commit a source-to-canonical adapter map.
Stream / normalize	Process the entire file in bounded memory. Convert price to cents, mileage to miles, numeric fields to canonical types and repeated categories to integer codes.
Validate / audit	Reject unusable price or malformed records under an explicit policy. Keep other missing preference fields as unknown. Log reason counts; never silently drop duplicates.
Materialize	Write compact search rows, vocabulary, UTF-8 details JSONL and uint64 detail offsets. Build custom B-tree snapshot separately from normalized rows.
Publish	Write temporary artifacts, flush, verify counts/versions and atomically publish. Manifest records SHA-256, source bytes, accepted/rejected counts and normalization version.

Proposed local storage contract

search.bin: fixed 32-byte records: price int64, year int16, mileage float32, five category uint16 codes, seating int16, four padding bytes and validity mask uint16. Assert itemsize=32 and round-trip the layout. Category code 0 is unknown; use validity bits for numeric fields and NaN for unknown mileage.

vocab.json: category codes/labels and reviewed source mappings. **details.jsonl + offsets.bin:** make/model, identity, display values and available URLs; final offset marks EOF. **btree/:** packed custom nodes, root and degree metadata. **manifest.json:** schema, scoring/data provenance and audit. Canonical schema is proposed until the CSV is inspected.

Memory and integrity

At 3 million rows, 32-byte search rows are about 96 MB; offsets about 24 MB. B-tree arrays, ranking workspace, details, interpreter and OS caching are additional. Memory mapping helps avoid eager copies but does not guarantee low RSS. Avoid millions of Pydantic models and full listing copies in heaps.

Release checks: source_rows = accepted_rows + rejected_rows; every accepted ID is retrievable; no sample flag or row limit. Synthetic/small development fixtures are allowed. Dataset changes invalidate indexes/caches. Missing or incompatible snapshots fail readiness with rebuild instructions, not an automatic CSV reload.

Filter, evaluate, then rank.

The tree handles price; the scorer handles the other eight preferences.

Custom B-tree - Anthony

Use minimum degree t (proposed t=32), with unique composite keys **(price_cents, record_id)** so duplicate-price cars survive. Non-root nodes have t-1 through 2t-1 keys; a node with k keys has k+1 children if internal; all leaves share a depth. A non-empty root has at least one key.

Implement root/child splitting before descent: split a full child, promote its median and update child references before selecting the next branch. Persist packed arrays and metadata after construction. The runtime index is immutable; deletion is outside this baseline.

```
BTreeIndex.build(prices, ids, min_degree=32)
BTreeIndex.range_ids(min_cents, max_cents) -> iterator[int]
BTreeIndex.save(path) / BTreeIndex.open(path)
BTreeIndex.validate() -> report
```

Range iteration prunes disjoint subtrees, includes both endpoints and returns qualifying **internal-node keys as well as leaf keys**, exactly once. Expected range cost: O(log N + C) key visits, with degree-dependent constants, for C candidates. Broad budgets can still return almost all cars. This is a B-tree, not a linked-leaf B+ tree.

Scorer and heap - Daniel

For every budget candidate, evaluate the eight conditions on page 3, form a bitmask and discard counts below six. Share one rank_key between every implementation. A custom heap queue exposes build, push, pop_best and len; implement bottom-up heapify, sift-up and sift-down. Keep record references or compact keys, not listing dictionaries.

Heapify included candidates, repeatedly pop, and cache a full ordered ID/mask sequence for pagination. For M included cars, complete extraction is O(M log M) with O(M) queue/result storage. A queue is not an in-place heap sort; do not claim O(1) auxiliary space for this design.

Open: assessed comparison pair

Proposed: custom heap-based complete ranking versus custom merge sort of the same candidates using the same key. Both must return identical full ordering. Conventional merge sort takes O(M log M) time and O(M) auxiliary space. Team adoption is deferred; decide before benchmark implementation. Neither library heapq nor library sort may replace an assessed implementation.

A simple scan and built-in sorted may serve as test oracles only. If the team instead chooses in-place heap sort, update the interfaces, ownership, memory claims and comparison protocol.

A shared API contract.

Proposed API v1; Pydantic validates the request and rejects unknown fields.

```
POST /api/v1/searches
{ "budget": {"min_cents": 1500000, "max_cents": 2500000},
  "preferences": {
    "year": {"min": 2018, "max": 2024}, "mileage_max": 60000,
    "body_types": ["suv"], "fuel_types": ["gasoline"],
    "transmissions": ["automatic"], "drivetrains": ["awd"],
    "seating_min": 5, "exterior_colors": ["black"]
  }}
202 {"search_id": "opaque-id", "status": "queued",
     "dataset_version": "manifest-hash", "scoring_version": 1}
```

Endpoint	Contract
GET /api/v1/health	Ready + dataset version/row count; 503 if prepared data is unavailable.
GET /api/v1/options	Known category codes/labels, observed bounds and make/model catalog; no raw listing dump.
GET /api/v1/searches/{id}	queued/running/completed/failed, stage, elapsed_ms; completion includes tier counts.
GET .../{id}/results	page=1, optional make/model; size fixed at 10. Returns page, pages, total, tier_counts and items.
GET .../{id}/facets	Available makes and alphabetic models from eligible results; optional make limits models.
GET /api/v1/listings/{record_id}	Display details. Search-specific explanations remain attached to the search.
DELETE /api/v1/searches/{id}	Cooperative cancellation at bounded loop checkpoints; clean partial files.

Result items: record_id, optional listing_id, make/model/year, price_cents, mileage, image_url/listing_url, matched_count, score_percent, tier, missed_preferences and unknown_preferences. Unknown also means missed. Counts follow refinement, before pagination; pages=ceil(total/10), or 0. Pages beyond the end return empty items.

Service orchestration and errors

Use one process and one search worker outside async routes; threads improve responsiveness, not CPU parallelism. Pipeline: price lookup, eight checks, retain 6-8, rank IDs, atomic cache, fetch ten details. Startup opens files through lifespan; do not rebuild on every launch.

Proposed cache: uint32 ranked IDs + uint8 masks, up to 15 MB/search at 3 million rows; maximum three completed searches; 15-minute inactivity TTL. Protect active reads from eviction. Errors use {code, message, fields, request_id}: 409 busy/not ready; 410 expired; 404 unknown; 422 invalid; 500 failed; 503 data unavailable. Preserve inputs and provide retry.

Readable folders. Repeatable setup.

Proposed commands below describe tools the team will implement.

```
daz-my-car/
  README.md, .env.example, .gitignore
  frontend/                 # package.json + lockfile
    src/app/                # shell, routes
    src/features/           # questionnaire/, results/
    src/components/ui/      # shared inputs, buttons, alerts
    src/api/, types/, styles/
    tests/
  backend/                  # pyproject.toml, requirements.lock
    app/main.py             # app factory, lifespan
    app/api/                # routes, schemas, errors
    app/services/           # search jobs, cache, coordination
    app/core/               # matching, shared rank key
    app/structures/         # btree, heap, comparator TBD
    app/data/               # codecs, store, source adapter
    tools/, tests/, benchmarks/
  data/raw/, prepared/, cache/    # local only; ignored
  docs/                     # PID, decisions, OpenAPI, reports
  .github/workflows/        # fixture-based checks
```

Toolchain and configuration

Pin a supported Node LTS compatible with the selected Vite release and a tested Python 3.12+ version. Bootstrap React TypeScript with Vite; follow Tailwind's Vite integration. Commit both dependency lockfiles. Backend dependencies: FastAPI, Uvicorn, pydantic-settings, NumPy; development: pytest, httpx, Ruff and a type checker.

```
# From backend/:
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.lock
python -m tools.inspect_csv --input /path/to/cars.csv
python -m tools.prepare_data --input /path/to/cars.csv \
  --output ../data/prepared/v1
python -m tools.build_index --data ../data/prepared/v1
python -m tools.verify_data --data ../data/prepared/v1
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
# Separate terminal, from frontend/:
npm ci
npm run dev -- --host 127.0.0.1
```

Document DATA_DIR, CACHE_DIR, RESULT_TTL_SECONDS=900, MAX_CACHED_SEARCHES=3 and MAX_ACTIVE_SEARCHES=1 in .env.example. Resolve paths explicitly. Proxy /api through Vite during development; direct cross-origin access requires explicit allowed origins. Serve built frontend and API from one origin for the local release.

README must include Windows equivalents, exact tested versions, disk-space needs, rebuild instructions and one complete health > options > search > next-page smoke test. Never commit raw CSV, caches, node_modules or .venv.

Integrate early. Verify the contract.

Each task ends with reviewable evidence, not just code.

Order / owner	Build task / exit check
1. All	Freeze request, scorer, rank key and fixture. Resolve assessed pair; assign comparator owner.
2. Anthony	Inspect schema, stream normalized files, build tree/snapshot. Manifest balances; store round-trip and range oracle pass.
3. Daniel	Implement eight-check mask and heap. Boundary/missing-field fixtures pass; full pop order matches reference.
4. Zachary	Build wizard/results against mock API, then real jobs. Typed HTTP client handles polling, cancellation, errors and stale responses.
5. Daniel + Zachary	Integrate job/cache/results/facets. Cross-tier pagination and full-set make/model refinements pass.
6. All	Prepare full data, profile M1, run comparison, review UI and rehearse handoff.

Frontend structure and behavior

Keep form state in reducer/context; keep active job, refinements and page separate. Stop polling on completion/failure/unmount; ignore late responses from older searches. Centralize fetch/error handling and API types. Provide visible labels/focus, keyboard support, responsive cards, reduced motion and image fallback; show real search state, not fake progress. Components: BudgetStep, PreferenceSteps, ReviewStep, ResultsPage, ResultCard, MatchBadge, TierHeading, MakeModelFilter, Pagination and ListingDetails. Do not duplicate authoritative scoring in React.

Required verification matrix

Area	Meaningful cases
Tree	Empty/singleton, multi-level splits, sorted/reverse/random inserts, duplicate prices, internal keys, inclusive/empty ranges, snapshot reload; verify invariants.
Matching / rank	8/8-6/8 retained, 5/8 rejected; strict budget; unknowns; range endpoints; category-set point count; ties and unknown mileage; full output equality.
Pages / API	0, 1, 9, 10, 11, 21 results; tier boundaries; no omissions/duplicates; refinement before slicing; invalid, busy, expired, cancelled and failed jobs.
Data / interface	Quoted multiline CSV, malformed rows, unit conversion, duplicate IDs, interrupted preparation; keyboard flow, input preservation, stale replies, image errors and empty states.

Measure the actual comparison.

Proposed protocol; no performance measurements exist yet.

The course asks for comparable implementations achieving the same task. If adopting heap-queue ranking versus merge sort, both must receive identical included candidates and return the identical complete order. Time heap construction plus full extraction against full merge sort. Shared preparation, tree filtering, scoring and serialization are outside the algorithm-only timer; report complete search latency separately.

Dimension	Protocol / evidence
Workloads	Release uses all prepared data. Fixed 100k, 500k and 1m subsets are additional scaling experiments only. Use narrow, medium and broad budgets and different preference selectivity.
Trials	One warm-up and at least five measured trials per workload. Alternate algorithm order; fresh identical input copies. Declare whether copying is timed; report median and range.
Correctness	Compare full ordered ID sequences before timing or compare full-output digests with samples. Top-ten equality alone is insufficient. Record failed trials and reasons.
Environment	Record commit, data checksum, CPU/RAM/OS, Python/dependencies, tree degree, input order, candidate count C and included count M.
Memory	Run each algorithm in a fresh process; capture peak RSS and baseline with consistent units. tracemalloc alone does not cover all native/NumPy allocations.
Interpretation	Both can have O(M log M) time yet differ in allocation, memory access and small-input overhead. The heap queue uses O(M), not in-place heap sort's O(1) auxiliary-space claim.

Proposed performance goals - validate on October 30

Representative broad-budget first search <= 10 seconds; cached-page API <= 500 ms; peak backend RSS <= 60% of installed RAM; full-data execution without memory failure. These are engineering targets, not promises. Confirm M1 RAM and revise targets from the first complete-data profile.

Report CSV preparation, index build, startup, first search and cached page separately. Remote image latency is separate. If targets fail: profile first; reduce duplicated objects/retained fields, improve compact access and persist indexes. Keep all valid listings. A compiled implementation is a later explicit decision if Python remains inadequate.

Release gate

All source records accounted for; no silent sample limit; matching and pagination verified; assessed pair selected and implemented from scratch; full output agreement; M1 profile recorded; fresh setup on a second machine; dataset/license reviewed; final report and video requirements confirmed.

A working release by November 6.

Proposed schedule in America/New_York; confirm the exact submission time.

Date	Team milestone / done evidence
Oct 27	All: adopt contracts; resolve comparison, schema, RAM and dependencies; commit fixture and repository skeleton.
Oct 28-29	Anthony: data/tree; Daniel: scorer/heap; Zachary: wizard/results. Each module passes fixture checks.
Oct 30	Full-data gate: complete manifest, tree query, peak memory and broad-budget profile on M1.
Oct 31-Nov 1	Integrated API, results, pagination, make/model refinement and details work end to end.
Nov 2-3	Comparator, benchmark trials, edge cases and UI error states complete.
Nov 4-5	Fresh-machine release candidate, full-data verification, docs, contribution record and video; freeze content.
Nov 6	Verify exact submitted artifact, tag known-good commit and submit before confirmed deadline.

Open decisions and risk owners

All, Oct 27: assessed pair and comparator owner; proposed tie rule and full-set make/model behavior; final submission checklist. **Anthony:** CSV path/real schema, source version/license, color mapping, image/link fields, RAM/disk. If capacity is tight, cut optional motion and extra features before correctness, comparison or full-data coverage. The course estimates 45 team hours; this scope is ambitious within that budget.

Handoff and maintenance

Keep main runnable with fixtures, use short reviewed branches and assign one owner/reviewer per issue. Each teammate must explain their implementation, including AI-assisted work. README contains setup, data provenance, tests and benchmarks. Update decision log, API examples and fixtures together; version scoring changes and invalidate incompatible caches.

Source basis and implementation references

Basis: supplied preliminary PDF, three assignment screenshots, conversation decisions and DESIGN-stripe.md. The design file guides this PDF's navy/indigo palette, gradient and light display hierarchy; locally available fonts are embedded. Actual CSV and benchmark results have not been inspected or measured.

Vite proxy/server options · FastAPI lifespan · FastAPI CORS · Python memory mapping
Vite setup · Tailwind Vite integration · Dataset reference from prebrief